

{EPITECH}

KUBE

< SAILING THROUGH THE CLOUDS />



KUBE

Welcome to an exhilarating journey into the world of cloud computing and containerization!
You are part of a dynamic team at a forward-thinking company, poised to explore the robust capabilities of Kubernetes.



Your mission is to revolutionize the company's containerization infrastructure, showcasing your skills in software engineering and system management.



Some tools are **imposed** (cluster provisioning with Ansible, native admission policy, OIDC authentication, HTTPS, private registry). Everywhere else, the tools below are **suggestions**: pick your stack and be ready to justify your choices.

Objectives

Your primary task is to design and deploy a fully-equipped Kubernetes cluster, not only to host and present an application, but also to embed its management tools and administration components.

Below are the goals you need to achieve, with some *tool suggestions* as starting points:

- ✓ expose the apps and services of your infrastructure through **Ingress or Gateway API** (*kubernetes-ingress* with Nginx/Traefik, or *Gateway API* with Traefik/Envoy Gateway, justified choice);
- ✓ a user-friendly **dashboard** to access and manage the containers (*kubernetes-dashboard* or *Headlamp*, justified choice);
- ✓ a **monitoring stack** to collect, visualize and **alert** on system and application metrics (*kube-prometheus* or *VictoriaMetrics*, justified choice);
- ✓ a **logging stack** to collect and visualize logs (*Loki*);
- ✓ a **GitOps** repository holding all these reusable component manifests (*kustomize*), reconciled by a **GitOps operator** (*ArgoCD* or *FluxCD*, justified choice).



After that, convert and deploy an application that was originally deployed using *docker-compose*:

- ✓ create a **Helm chart** for this application, and use the official chart for the database (*helm*);
- ✓ create a **GitOps** repository to deploy this application (*kustomize*);
- ✓ **automate** the deployment and **verify the application status** through your GitOps operator (sync and health).



You can find the provided application on My, in the module.

Best practices

During this experimentation, implement all the "best practices" of a Kubernetes environment:

- ✓ define resource **limits and requests** on CPU and RAM consumption;
- ✓ use **Secrets** to hold sensitive data, with **no plaintext secret committed to Git** — use *Sealed Secrets* or *External Secrets* (justified choice);
- ✓ store sensitive data in **Secrets** and all other configuration in **ConfigMaps**;
- ✓ **label** all your resources, following Kubernetes recommendations;
- ✓ add **redundancy** to your application, with multiple replicas and affinity rules, at least for the "application" containers (replicas of a service must run on different nodes);
- ✓ add **persistent storage** for your database: the data must persist across database pod restarts (surviving a full cluster deletion is not required), using a `PersistentVolume` backed by the **shared EFS file system provided to your group**;
- ✓ schedule a **CronJob** that answers a real operational need (volume backup, maintenance task, scheduled report);
- ✓ serve all exposed endpoints over **HTTPS** (TLS via *cert-manager* and *Let's Encrypt*);
- ✓ publish your application image to a **private authenticated registry**, and pull images from it.

Security and control

You will also have to add security and control:

- ✓ add a **native admission policy** (`ValidatingAdmissionPolicy`, CEL-based) that validates and controls requests — for example enforcing resource limits, mandatory labels, or an allowed image registry;
- ✓ add **authentication** through **OIDC** (via *dex*), covering **all the tools you deploy** (Grafana, dashboard/Headlamp, ArgoCD, kubectl, ...) **and** the Kubernetes API, against an **identity provider you deploy yourselves** in your cluster (e.g. *Keycloak*), with a justified choice.



The lab provides no corporate directory for this: the identity provider is a workload of your own, like any other component you deploy. Your instances have Internet access, so nothing external is needed. Plan for it early — it is dex's federation, and the coverage of every tool plus the Kubernetes API, that gets evaluated.

Infrastructure



Each team is provided with dedicated compute resources hosted on **Amazon Web Services (AWS)**:

- ✓ kube-1 — Kubernetes node (control plane + worker);
- ✓ kube-2, kube-3 — Kubernetes worker nodes.

kube-1 acts as the control plane, while the worker nodes provide compute capacity. At least two workers are available so that high availability and affinity rules can be demonstrated.

Exposure relies on an Ingress controller reachable through NodePort, resolved via **sslip.io** (or **nip.io**) rather than a dedicated load balancer or wildcard DNS record. Your **kube-1 instance carries a stable public IP address** that survives the nightly shutdowns: it is your entry point, and what keeps your hostnames and TLS certificates valid from one day to the next. The two workers have no fixed public IP — theirs changes on every restart — so do not build anything on them from the outside; their private addresses inside the VPC, on the other hand, are stable, so the links between your nodes hold. A shared **Amazon EFS file system** is provided to your group and can be mounted on your instances: use it to back your persistent storage. VM access is provided through **AWS SSM Session Manager** (AWS console or AWS CLI) instead of a VPN or public SSH — you sign in through the IAM Identity Center portal with your school email address, with no SSH key to manage. Once you are in, you are free to deploy your own SSH keys on top if that suits your provisioning better.

You must **provision and deploy the cluster with Ansible**, so that it can be torn down and rebuilt reproducibly.

Keep a clear separation between two scopes: the **infrastructure** (cluster, components, observability, security) and the **application** (the converted app). Each scope has its own GitOps repository.

Please note that all virtual machines are automatically shut down every evening to conserve resources and minimize costs. However, you can restart them at any time if needed.



Delivery

Push your configurations into two repositories: one for the infrastructure components, one for the application.

Your Ansible playbooks must be included. Documentation is mandatory, especially the cluster provisioning, the container setup steps and the commands.



Defense

You are expected to present:

1. the cluster creation and configuration process, clearly showing the responsibility of each component and the deployment of each building block;
2. the architecture of your infrastructure;
3. the GitOps application deployment process.

The live demonstration must include:

- ✓ provisioning a fresh cluster with Ansible;
- ✓ deploying a **new version** of the application through your GitOps operator (a Git commit reconciled by the operator, or simple deployment commands) — not a manual first-time setup;
- ✓ a **progressive deployment**: the application stays available throughout the rollout, and a faulty new version does not take over — the previous version keeps serving (readiness probes and a rollout strategy that keeps the previous version, optionally a canary/rollback tool);
- ✓ evidence that authentication (OIDC), HTTPS and the admission policy are enforced.



Enrich the new version with a fault (for example a failing readiness probe) so you can show that the rollout does not switch traffic to a non-functional version.

Bonuses

You may also want to add some extras, such as:

- ✓ multi-tenancy and authentication for *Prometheus* and *Loki*, via *kube-rbac-proxy*;
- ✓ an evaluation of Helm artefact security;
- ✓ definition of fine-grained orchestrator access permissions (RBAC);
- ✓ reinforcement of network access (subject to the limitations of the existing infrastructure);
- ✓ lightweight Docker images;
- ✓ zero-downtime deployment (successful HTTP requests during deployment);
- ✓ documentation of the cluster upgrade process (version upgrade, node drain, component updates);



v 2.0.0

{EPITECH}